AHA Intern Capstone: Patient Portal & AI Appointment Assistant

30-Day Project Plan for David, Covenant and Oluwatoyin

1. The Use Case

Problem: Booking a health appointment usually means calling a receptionist, filling out a form, or fighting with a clunky patient portal. Patients also can't easily see their own records, and staff lose time on manual scheduling and data entry.

Idea: A small web app where patients log in, see only their own records, and talk to an AI chat widget to book or cancel appointments and check what's coming up. No forms needed. It combines data privacy (only you see your records) with AI convenience (booking feels like texting a receptionist).

This one idea splits naturally into three areas the interns can each learn from (software engineering, data and security, and design) while still shipping one shared product.

Scope for the time available (MVP):

- Seeded synthetic users and fixed appointment slots, so there is no real registration flow or email.
- Patients view their own records (read-only) and book, cancel and list appointments by chat.
- Staff add records and see today's schedule. Editing records is a stretch.
- The backend provides a small staff view of appointments per day and busiest slots. Machine learning stays a stretch goal.

2. Roles & Permissions

Role	Can do
Patient	Log in, view only their own records, and chat with the AI bot to book, cancel or check appointments
Medical staff	Log in, add records for any patient, and view today's full appointment schedule

One rule protects both roles: the AI never decides who the user is. The backend passes in the logged-in user, and every action is checked in code, not in the prompt.

3. Core Data Model

A simple starting point the interns can refine as they go:

- **users:** id, name, email, password_hash, role (patient or staff)
- **records:** id, patient_id, staff_id (who wrote it), notes, date created
- **appointments:** id, patient_id, staff_id, date/time, status (booked or cancelled)

All data must be made up. No real patient information at any point.

4. Team Split

Intern	Main focus	Owns
David	Software engineering	Backend logic, APIs, login flow, connecting the chat widget to the backend, making it all actually run
Covenant	Data and security	Database design, password hashing, role-based access control, protecting patient data, testing the system for loopholes
Oluwatoyin	Design	Patient and staff screens, the chat widget's look and feel, dashboards, making the whole thing easy to understand

Each intern still touches the whole app. This is about who leads each part, not working in silos. You three will need to talk to each other and collaborate the whole way through. David needs Oluwatoyin's screens to build against, and Covenant needs to check David's work for gaps.

Support: the backend devs provide a working login and role-check skeleton for David and Covenant to extend. The AI/ML mentor covers the AI API, tool calling and AI security. The design team supports Oluwatoyin.

5. Individual Tracks

David: Software Engineering

What he's building: the working app, the part that makes everything run.

- Set up the backend (his choice of framework, such as Flask or Express).
- Build the three data tables and basic create, read and update actions.
- Extend the login skeleton from the backend devs, using the hashing approach Covenant sets up.
- Build the check that decides what a logged-in user is allowed to see.
- Wire the chat widget to the backend: patient sends a message, the message goes to an AI API, the model asks for a function (book, cancel or list), and the backend checks it and runs it for the logged-in user only.
- Connect the backend to Oluwatoyin's screens.

Skills covered: APIs, databases, login flow, hooking a screen up to a backend, calling an outside AI service, tool calling, and debugging when pieces don't talk to each other.

Stretch goal: a small staff view of appointments per day and busiest slots, as a taste of data analysis. Machine learning is optional after that.

Learning resources:

1. First AI API call: Dataquest, Build an AI Chatbot with Python and the OpenAI API. dataquest.io/blog/build-an-ai-chatbot-with-python-and-the-openai-api
2. Tool calling (shared with Covenant): see the resources listed under Covenant.
3. Backend: from the backend devs.
4. Stretch: freeCodeCamp, Data Analysis with Python youtube.com/watch?v=r-uOLxNrNk8 and StatQuest, A Gentle Introduction to Machine Learning youtube.com/watch?v=Gv9_4yMHFhI

Covenant: Data & Security

What she's building: the trust layer underneath the app.

- Work with David to finalize the database design: what fields exist, how tables relate, what's sensitive.
- Set up password hashing so nothing is ever stored in plain text.
- Design and test the access rules. Can a patient see someone else's records by changing a link or a request? Can staff accidentally edit the wrong patient?
- Think through the chat widget's specific risks: could a patient trick the bot into pulling up someone else's appointments, what should the bot never be allowed to do (like adding a medical record), and what happens if the AI misreads the request?
- Test that the patient's identity always comes from the login session, never from the chat text.
- Write a short security review: what's protected, how, and what would still worry her with more time.
- Try to break the team's access rules once the app is built.

Skills covered: the difference between login and permissions, data privacy, AI-specific risks such as prompt injection and hallucinations, writing up findings clearly, and testing driven by real questions rather than guesswork.

Learning resources:

1. Foundation: freeCodeCamp, Learn Prompt Engineering freecodecamp.org/news/learn-prompt-engineering-full-course with the short "How LLMs work" explainer youtube.com/watch?v=FCMNnOQiG-k
2. Tool calling primer, with a short video on how a model asks for a function: docs.retellai.com/integrate-llm/integrate-function-calling and youtube.com/watch?v=9qyML06RiT0
3. Tool calling in an appointment assistant (use the tool definition and agent parts, skip Cal.com and deployment): docs.cerebrium.ai/v4/examples/langchain-langsmith
4. Reference: platform.openai.com/docs/guides/function-calling
5. Hands-on prompt injection: Lakera's "Gandalf" game, then attack the team's own chat widget.

Oluwatoyin: Design

What she's building: how the app looks, feels, and is understood.

- Sketch both experiences: the patient view (records plus chat widget) and the staff view (adding records, today's schedule).
- Design the chat widget so it feels like a real conversation, not a form pretending to be one.
- Design how records are shown so a patient with no technical background understands them at a glance.
- Put together a simple dashboard idea for staff: today's appointments, easy to scan.
- Decide what should be visible right away versus hidden until clicked, the design side of privacy.
- Turn the sketches into real screens (HTML/CSS, or Figma turned into basic HTML) that David can connect his backend to.

Skills covered: designing for sensitive information, organizing what matters most on a screen, accessibility basics, turning a sketch into something buildable, and treating privacy as a design choice, not just a backend rule.

Learning resources: provided by the design team.

6. 30-Day Timeline

Week 1: Understand & Plan

- **All three:** agree on the data model and what David's backend will send and receive (the API contract), so the others can plan around it.
- **David:** set up the project and an empty database.
- **Covenant:** draft the schema and access rules; start researching AI-specific security risks and learn how tool calling works.
- **Oluwatoyin:** sketch the patient and staff screens; check with David and Covenant that they're realistic to build.

Week 2: Build the Foundation

- **David:** build login, the permissions check, and basic actions for records and appointments, extending the backend devs' skeleton.
- **Covenant:** build password hashing with David; write test cases meant to break the permissions; start the security write-up.
- **Oluwatoyin:** turn the sketches into real Figma or HTML/CSS screens for both roles.

Week 3: Bring It to Life

- **David:** connect the chat widget to an AI API; turn its replies into booking, cancel and list actions; hook up Oluwatoyin's screens.
- **Covenant:** test the chat widget specifically for tricks; finish the access rules; help David fix any login bugs found.
- **Oluwatoyin:** adjust the chat widget's look once it's actually working; polish the records view and staff dashboard.

Week 4: Harden, Polish, Present

- **All three:** test the whole thing together. One plays patient, one plays staff, one tries to break it.
- **Covenant:** finish the security write-up.
- **Oluwatoyin:** final visual polish so the demo looks clean.
- **David:** fix whatever testing turns up; add the staff insight view if time allows; prepare the technical walkthrough.
- **All three:** rehearse the shared demo and each person's 10-minute talk.

7. Final Presentation (10 minutes each, same structure)

Each intern presents using the same structure from the internship plan, applied to their own part:

1. **The Problem:** what healthcare problem this addresses.
2. **Why It Matters:** who's affected (patients, staff).
3. **What I Researched:** what they learned specific to their part.
4. **Our Solution:** what they built or decided.
5. **Our Prototype:** a live look at their piece.
6. **Technology Behind It:** how it works, explained simply.
7. **What I Learned.**
8. **What's Next:** what they'd improve with another 30 days.

Suggestion: start with one shared live demo, where a patient books an appointment through the chat and it shows up on staff's schedule, then let each intern speak to their own angle on it.

8. Success Criteria

By the end, the team should be able to show:

- A patient can log in, see only their own records, and book or cancel an appointment by chatting with the AI widget.
- A staff member can log in, add a patient's record, and see today's schedule.
- Covenant can explain what's protected and how, and point to at least one thing she specifically tested for, such as a patient failing to reach another patient's records through the chat.
- Oluwatoyin can explain the design choices that make the app clear and trustworthy to use.
- David can explain how the pieces (screens, backend, database and AI API) talk to each other.

The goal isn't a polished, production-ready app. It's each intern walking away able to say: "This is the kind of technology I enjoy, this is the problem I care about, and this is what I want to learn next."